CSC111 Project 2: Constellation Explorer: Learning Sky Patterns Through Interactive Graphs

Minsung Kim, Arish Zobeyer, Ryder Lin, Justin Chan

April 2, 2026

1. Project Title and Group Members

Project title: Constellation Explorer: Learning Sky Patterns Through Interactive Graphs

Names of all group members: Minsung Kim, Arish Zobeyer, Ryder Lin, Justin Chan

2. Introduction and project goal

When beginning to learn about constellations, learners tend to make the same mistake. They may have great name recognition for several well-known constellations, like Orion, Ursa Major, or Cassiopeia, but when they look up into actual starlit skies to find familiar constellational patterns, it is a lot tougher. Usually, constellations are illustrated as basic stick figures in textbooks or diagrams. However, a real starlit sky contains so much more than the stars in books that comprise each constellation; multiple stars with varying brightness levels also comprise each constellation in reality. Therefore, being familiar with constellation figures does not guarantee that we can recognize every constellation in the nighttime sky.

It was this gap of knowing a constellation's shape and actually being able to identify a constellation in the nighttime sky that motivated our project. We wanted to create a way for people to practice their constellation recognition skills and their memorization skills.

We developed an interactive star map activity in which users will attempt to recreate a star constellation by clicking on stars in the star map. Upon completion of clicking selected stars by the user, the program will calculate how closely the user's selected stars resemble the actual star pattern of the identified constellation. This will provide the user with immediate and visible feedback to use in determining the accuracy of their selection of stars to create the constellation. Using this interactive star map activity multiple times with different constellations will allow the user to gradually improve their ability to recognize star constellations in the nighttime sky.

How can we strengthen the average person's knowledge of constellations without requiring them to memorize them?

By encouraging active recall and shape matching, users identify stars and link them together to reconstruct familiar patterns. In doing so, users will develop "constellation intuition" that a static set of notes alone will not be able to provide. Contextual and naming references for the constellations are drawn from commonly used educational astronomy resources, including Glyphweb's constellations resources[1].

[1] Glyphweb, "ESky: Constellations," `https://www.glyphweb.com/esky/constellations/`.

3. Datasets

There are three CSV files in `project2_datasets.zip`: `stars.csv` gives each star's sky position and brightness; `asterisms.csv` gives the official line segments (which stars connect for each constellation); and `centered_constellations.csv` gives extra metadata (names, meanings, etc.) to label and centre each constellation in the view. The primary source these tables are derived from or aligned with is Eleanor Lutz's Western Constellations Atlas of Space on GitHub[2].

The following is an overview of `data/stars.csv` (star catalog, used at runtime):

- Purpose: One row per star used in the constellation view.
- The fields used by our code:
  - **hip:** Hipparcos ID, which is a primary key in our graph of data type integer.
  - **ra:** Right ascension in hours, converted to degrees when loaded.
  - **dec:** Declination in degrees.
  - **mag:** Apparent magnitude, used for star size on screen.
  - **proper:** Common name shown if available.
- Note: If `stars.csv` is missing, it can be regenerated from `data/asterisms.csv` using `process_stars.py` in the same folder as `main.py` (not under `data/`). For grading, `stars.csv` is already in the zip, so you do not need to run that script.

The following is an overview of `data/asterisms.csv` (constellation line structure):

- Purpose: determine how stars are connected for the purpose of forming each constellation figure.
- The fields used by our code: `constellation`, `stars`, coordinate lists.
- Processing details for `constellations.py`: parses the encoded list values using ast.literal eval, adds vertices/edges into graph, builds a membership map from constellation code to HIP ids.

The following is an overview of `data/centered_constellations.csv` (metadata):

- Purpose: provide human-readable names and descriptive context for HUD.
- The fields used by our code: `constellation`, `name`, `name_meaning`, `class`, `ra`, `dec`.
- Role in program: displays constellation title/meaning and supports initialization of centered view.

The data set was purposely designed to be as small as possible to minimize file size, thereby making it simple and reproducible.

[2] Eleanor Lutz, *Western Constellations Atlas of Space*, GitHub repository, `https://github.com/eleanorlutz/western_constellations_atlas_of_space`.

4. Computational overview

4.1 Representation of data: the fundamental model of graphs

The project's use of stellar images and constellations is, in fact, a function of the graph model.

- In `graph.py`, the class `WeightedGraph` has the stars represented as vertices, with four attributes (RA, Dec, magnitude, name) and undirected edges connecting two stars if they are adjacent to one another based upon the definition of their constellation line.
- In `constellations.py`, when the method `load_constellation_graph()` is executed, it creates the graph using the constellation data in CSV format. Then it returns both the graph and a dictionary mapping each constellation code to the set of HIP star IDs in that constellation.
- In `similarity.py`, we use the class `ConstellationGraph` to score a user's drawing against the constellation graph. We construct the user's drawing and the actual constellation as two separate graphs, and we use them to compare similarity.

Thus, trees and graphs are the primary form of representation used throughout the loading, matching, and scoring processes.

4.2 Main computations and algorithms

Algorithm 1: Loading and preprocessing

- Parse the star records to generate a complete list of vertices.
- Parse the edge lists for each constellation to generate undirected edges.
- Create the code-to-stars mapping to have fast access for visual rendering.
- Load the constellation data which contains the constellation name, meaning, and centered coordinates.

Algorithm 2: Building the user graph from interaction

- The path is the order of stars clicked by the user.
- The edges are created between consecutive selected stars in the user graph.
- Clicking the last star again removes the most recent edge from the user graph. (Undo)

Algorithm 3: Similarity scoring (implemented in `similarity.py`)

- Normalize the two objects so that when compared there is no sensitivity to their absolute dimensionality.
- Rotational alignment: Find the best orientation to align the reference object to the user's object.
- Structural Score: Compare the connections between the two graphs (number of edge intersections, number of missing edges, and number of additional edges present).

- Geometric score: Compare how the edge of the object was positioned relative to the edge (including length of the edge).
- Final output: combine components and convert to integer percentage via `score_to_percentage()`.

The final output will combine the three components and represent them as an integer percent. The scores will be converted to percent with `score_to_percentage()`.

We chose the above program outline as the user is not required to click accurately, and they will receive recognition for their effort if their total shape is close enough.

Algorithm 4: Additional algorithms We also implemented `star_hopping.py`, which is another module that uses weighted graphs to form hop-edges between nearby stars and solves shortest-path calculations with Dijkstra's algorithm. This will produce meaningful graph calculations.

4.3 Visual and interactive reporting

The program visually conveys computational outcomes in real time through a pygame window, including:

- The background star map.
- Reference constellation lines, colored grey.
- User-drawn lines, colored cyan
- HUD labels (constellation name, meaning)
- The HUD will contain a label of the constellation name, its meaning and provide a final similarity score once Enter has been pressed.

In `game.py`, the interaction loop encompasses the handling of keyboard and mouse events in conjunction with redrawing all frames. Rendering helper functions located in `visualization.py` will provide support for projecting RA/Dec to screen coordinates, determining star radius based on magnitude, snapping to the closest star and rendering text on the screen.

4.4 External libraries

- **pygame**: display creation, event handling, drawing primitives, and font rendering. We used pygame to support interactive constellation reconstruction.
- **python-ta**: used during development for code style and quality checks, helps readability and maintainability.

5. Instructions for obtaining datasets and running the program

1: Install Python and dependencies

Use Python 3.13.5. In the project folder run:

```
pip install -r requirements.txt
```

2: Obtain datasets

Download `project2_datasets.zip` from MarkUs and unzip it so that `data/` is beside `main.py`. The expected layout is of the following:

```
<submission folder>/
  main.py
  game.py
  visualization.py
  constellations.py
  similarity.py
  graph.py
  requirements.txt
  data/
    stars.csv
    asterisms.csv
    centered_constellations.csv
```

3: Run the program

```
python main.py
```

Optional (start directly in star-hop mode):

```
python main.py --hop
```

The program will look like this:

A window named "Constellation Explorer," will display stars represented by white/yellow dots. Brighter stars will have larger representations. A target constellation is loaded into the application, and it will have faint line references. Users will be able to reconstruct the target constellation by clicking on the stars and then pressing Enter to obtain a percentage similarity score.

Controls

- **Left-click stars:** Draw lines between selected stars
- **Right-click:** Lift the pen and start a new stroke
- **Click last star or press Z:** Undo the last point
- **Press Enter:** Score your drawing
- **Press R:** Reset the drawing
- **Press G:** Toggle the reference constellation graph
- **Press N:** Load the next random constellation
- **Press Q or close window:** Quit the program

Screenshots of results after running program

Figure 1: After running program

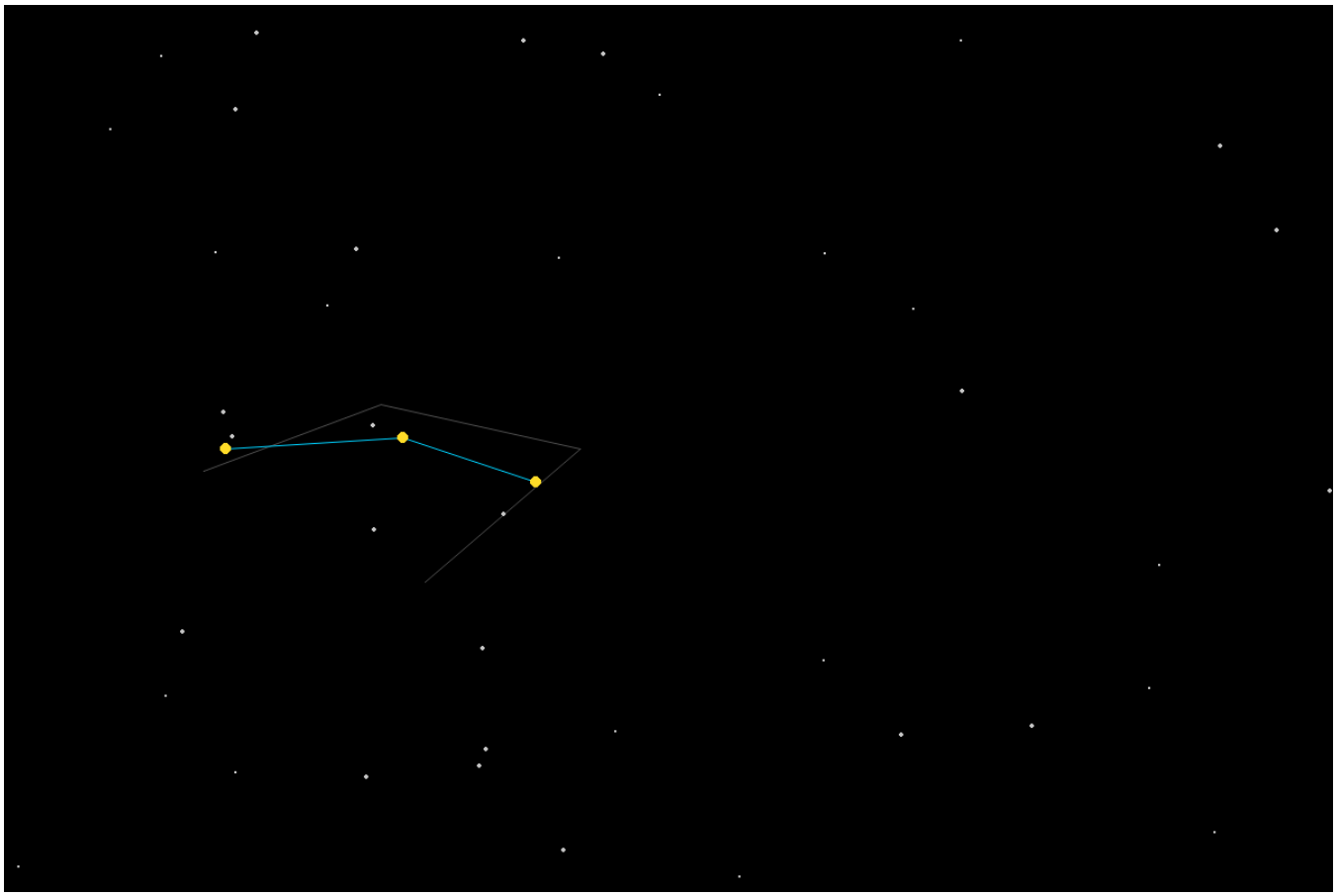

Figure 2: After left clicks to draw constellation path

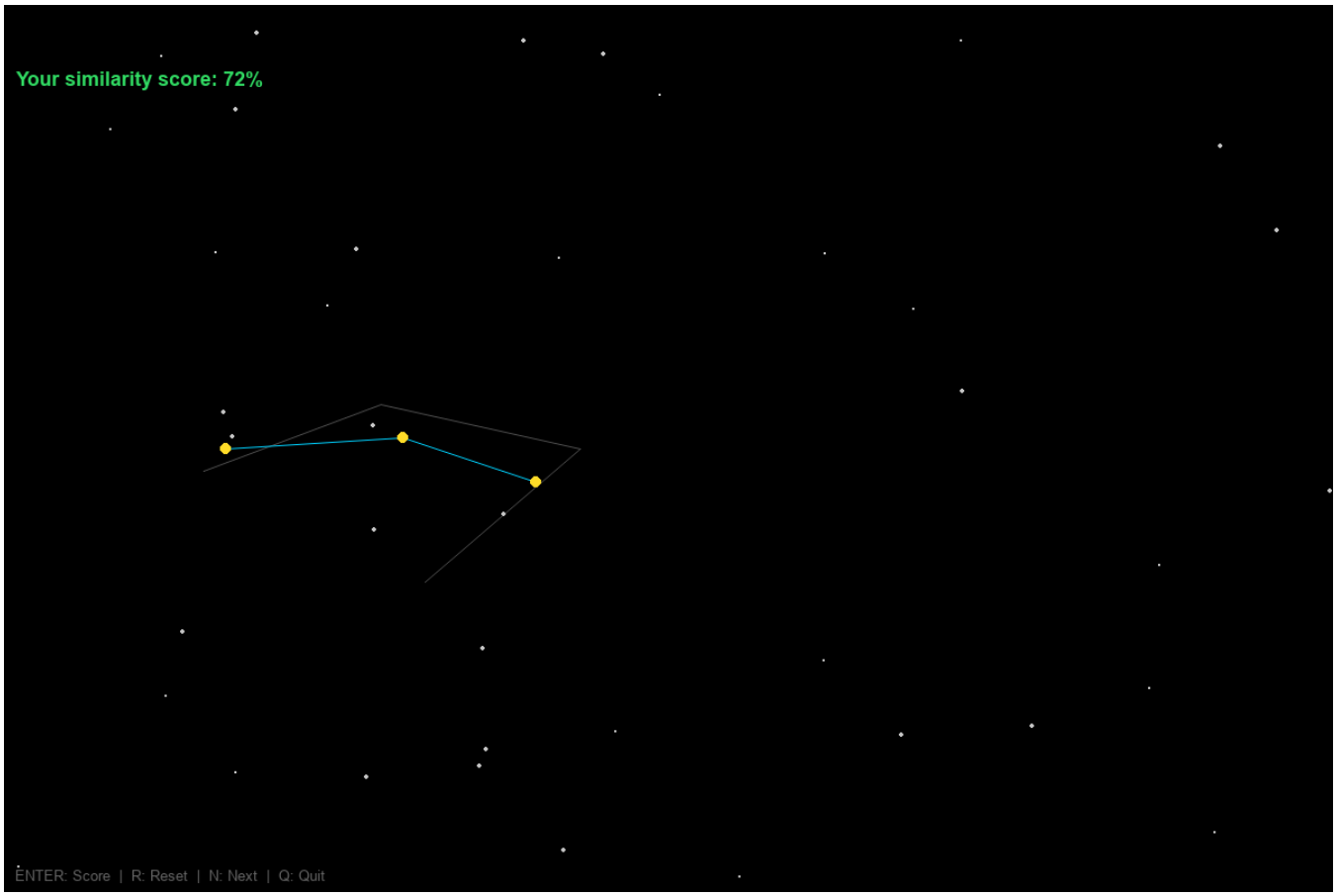

Figure 3: After pressing Enter to check similarity score

6. Changes to our project plan

The final implementation was close to the concept we envisioned at the beginning of this project, but had a few significant alterations in terms of the actual decisions that were made while implementing it.

1: Scoring methodology

The original proposal identified that scoring would be accomplished using a recursive and backtracking methodology (over a graph) and that we would utilize this same methodology to develop a scoring technique for how users would click. However, it became clear while completing the implementation of the project that simply utilizing a pure graph structure would limit our ability to create a scoring methodology that fits well with how people click. As a result, we transitioned from creating a pure graph-based scoring methodology to a more mixed scoring technique using both numerical normalization and rotational alignment, in addition to utilizing the structure of the graph, resulting in a composite structural/geometric-based scoring method.

As a result of this change, we expect to provide users with more intuitive results when they have made minor errors in either ordering or positioning of stars.

2: Interactive modes

We originally discussed a second interactive mode for star hopping. We completed the underlying algorithmic module (`star_hopping.py`) and integrated it into the pygame interface as a second mode (toggle with key H, or start in hop mode using `python main.py --hop`). In hop mode, users click a START and TARGET star, and the program highlights an easiest hop route computed by Dijkstra's algorithm.

3: Tools used during implementation

Our initial planning iterations suggested that we would use NumPy; however, due to our eventual decision to use only the Python standard math library and corresponding Python standard data structure library in the final run-time path, we significantly reduced setup complexity & prepared our project with minimal dependencies.

4: Dataset naming and packaging

In our internal drafts, we had generic file names like "constellation lines.csv". Our final implementation consistently uses `asterisms.csv`, `stars.csv`, and `centered_constellations.csv`. This also allowed us to more formally demonstrate reproducibility.

7. Discussion

We aimed to create a constellation learning experience with less dependence on memorization than typically occurs in educational situations. From the output of the program, we feel that we have succeeded in providing learners with an experience where they can actually practice rather than just completing an educational study. Users receive instant feedback via their score, have the opportunity to retry instantly, and can move through constellations with one key press. In practical terms, this allows them to develop their visual pattern recognition skills through repeated practice, which is the major area of struggle for beginner learners transitioning from using diagrams to identifying constellations in actual star field observations.

One of the key aspects of our project is the relationship between the computational function and the user. The graph model is not simply behind the scenes; it is the basis for data loading, rendering and scoring all visualization efforts. When a user presses the "Enter" key, the result is based on both the structure of the graph model (what stars are connected) and the geometry of the visual object on the screen (where it appears in relation to the graphic image of the stars). The combination of these factors makes the feedback users receive feel more credible than just determining whether or not their answer was correct.

For example, learners will receive partial credit when they correctly create a visual object that represents the overall form of the constellation, but do not connect one or more of the stars.

Reproducibility is one additional positive outcome. The data bundle, along with all its dependencies, is small in size, with a simple entry point required to interact with the application. Therefore, anyone has immediate access to a working version of the program.

Having explained some positive aspects of this application, limitations must also be recognized.

First, there are projection limitations. We are using a very simple linear-based projection technique to convert the RA/Dec of stars into the respective x/y pixel coordinates on the screen. This is certainly acceptable for a simple 2D educational interface, but it will not yield astronomically accurate results across larger regions of the night sky. By moving to a more sophisticated projection technique, we may improve the geometric accuracy of the stars' arrangement.

Second, semantic interaction limitations create issues for users. User touch inputs are recorded as continuous single-click actions, which join a new line from the last selection to form a constellation. As our app does not support drawing or tracing multiple branches, users may find it hard to visualize the stars in the constellation as being connected through many branches and may be forced to visualize it as a single stroke connecting branches together.

Third, score sensitivity is still a limitation. Even with normalization and rotation alignment, the final score can fluctuate depending on how much weight we give to geometric shape versus graph structure. This can be seen clearly when we have constellations with few edges, and that is when adding or removing a connection leads to a large fluctuation in score.

Fourth, this program uses datasets that is from a narrow cultural scope. The mappings of stars into physical space are derived from one atlas. Therefore, they are representative of one cultural understanding of the stars. This can introduce some bias, so we cannot consider it universal.

Fifth, some limitations may also exist in the star-hopping models. The star-hopping Dijkstra-based model functions properly but has a fixed angular threshold and weighting difficulty for the star hops. These parameters need adjusting to better capture variances in different use cases, while enhancing the model with more realistic use-case conditions.

If we continue the project, our priority next steps would be refining hop weights, such as improving path explanations and hop weights. Simultaneously, we should add optional hints to users, like providing the next likely star. Also, we can run small user group tests to measure the gain in users' knowledge, and update the score weight based on users' activity.

In summary, we believe that our program meets our project main goals clearly: it provides an interactive way for users to learn to recognize constellations. It also provides users with numerical feedback and allows for enhanced learning through experience. Overall, the program is an ideal, strong foundation that combines graphical models, non-trivial computing, and user interaction.

8. References

- Eleanor Lutz. *Western Constellations Atlas of Space*. GitHub, 2025. `https://github.com/eleanorlutz/western_constellations_atlas_of_space`.
- Glyphweb. "ESky: Constellations." `https://www.glyphweb.com/esky/constellations/`.
- Pygame Developers. *Pygame Documentation*. `https://www.pygame.org/docs/`.

- Read the Docs. "Introduction to Pygame." https://pygame.readthedocs.io/en/latest/1_intro/intro.html.
- University of Toronto Writing Advice. "Using Sources / Documentation." https://advice.writing.utoronto.ca/using-sources/documentation/.
- Python-TA project. https://github.com/pyta-uoft/python-ta.